

Graph Convolutional Network for Dendritic Tree Classification

S1. Graph Representation

Each SWC reconstruction was represented as a collection of individual dendritic trees originating from the soma. Every dendritic tree was treated as an independent graph $G = (V, E)$, where nodes correspond to neuronal compartments and edges represent parent-child relationships extracted directly from the SWC file.

The soma was included in every graph to preserve the anatomical root of each dendritic tree. Since graph convolutions benefit from bidirectional information flow, every parent-child edge was represented in both directions. Consequently, if an edge existed between nodes u and v , two directed edges (u, v) and (v, u) were included in the graph.

For every node, a five-dimensional feature vector was constructed:

$$\mathbf{x}_i = [\text{node type}, x_{rel}, y_{rel}, z_{rel}, \text{radius}]$$

where

- **node type** corresponds to the SWC compartment type,
- $x_{rel}, y_{rel}, z_{rel}$ are coordinates relative to the soma,
- **radius** is the compartment radius.

To reduce variability caused by neuron size and absolute position, the spatial coordinates and radius were standardized independently within each dendritic tree by subtracting the mean and dividing by the standard deviation. The categorical node type was not normalized.

S2. Cumulative Sholl Feature

In addition to graph topology, a global morphometric descriptor was incorporated using cumulative Sholl analysis.

For each dendritic tree, branch points and terminal nodes were identified. The path from the soma to each target node was reconstructed, and Euclidean distances from the soma were computed along the path. A Sholl value was then calculated by counting intersections within successive radial shells, where the shell width was defined as 10% of the total path length.

The cumulative Sholl value for a tree was obtained by summing the Sholl values across all branch points and terminal nodes:

$$S = \sum_{k=1}^N s_k$$

where s_k denotes the Sholl value associated with target node k .

This cumulative Sholl measurement provided a global descriptor of dendritic complexity that complemented the local structural information learned by the graph convolutional network.

S3. Graph Convolutional Network Architecture

The proposed network consists of two graph convolutional layers followed by a separate fully connected branch that processes the cumulative Sholl feature (Figure S1).

The network architecture is summarized below.

Component	Configuration
Input node features	5 features per node
Hidden representation	Two GCN blocks, each with 256 hidden units (Graph Convolution → Batch Normalization → ReLU; dropout of 0.4 after the first block)
Graph pooling	Global mean pooling
Sholl branch	Fully connected layer with 256 hidden units and ReLU activation
Feature fusion	Concatenation of graph and Sholl embeddings (512 dimensions)
Output layer	Fully connected classifier with three output neurons corresponding to apical, basal, and other classes

The final output layer produced logits corresponding to the three dendritic classes:

- apical
- basal
- other

S4. Graph Convolution

Each graph convolutional layer followed the normalized message-passing formulation introduced by Kipf and Welling:

$$H^{(l+1)} = \sigma(\hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2} H^{(l)} W^{(l)})$$

where

- $H^{(l)}$ denotes node features at layer l ,
- $W^{(l)}$ is the trainable weight matrix,
- $\hat{A} = A + I$ is the adjacency matrix with self-loops,
- $\hat{D}$ is the corresponding degree matrix,
- $\sigma(\cdot)$ denotes the ReLU activation function.

Self-loops were added to every node to preserve each node's own features during message passing.

The normalized adjacency matrix ensured that information from neighboring nodes was aggregated without bias toward highly connected nodes.

S5. Neighborhood Aggregation and Feature Transformation

For every node, neighboring feature vectors were aggregated using degree-normalized weighted summation:

$$h'_i = \sum_{j \in \mathcal{N}(i)} \frac{1}{\sqrt{d_i d_j}} h_j$$

where

- $\mathcal{N}(i)$ denotes neighboring nodes,
- d_i and d_j denote node degrees.

The aggregated neighborhood representation was then transformed using a learnable linear projection,

$$z_i = W h'_i + b$$

followed by

- Batch Normalization,
- ReLU activation,
- Dropout after the first graph convolution layer.

This sequence was repeated for the second graph convolution layer.

S6. Graph-Level Representation

Following the second graph convolution layer, node embeddings were converted into a graph-level representation using global mean pooling:

$$g = \frac{1}{|V|} \sum_{i=1}^{|V|} h_i$$

where h_i denotes the final embedding of node i .

The cumulative Sholl value was processed independently through a fully connected layer with ReLU activation to generate a second 256-dimensional embedding.

The pooled graph embedding and Sholl embedding were concatenated,

$$z = [g; s]$$

forming a 512-dimensional representation that served as input to the final classifier.

S7. Training Procedure

The model was implemented in Python using PyTorch and trained using the AdamW optimizer with the following hyperparameters:

Parameter	Value
Optimizer	AdamW
Initial learning rate	5×10^{-5}

Parameter	Value
Weight decay	1×10^{-4}
Loss function	Cross-Entropy Loss
Learning-rate scheduler	ReduceLROnPlateau
Scheduler factor	0.5
Scheduler patience	5 epochs
Epochs per training cycle	50
Training cycles	10
Hidden dimension	256
Dropout	0.4

The model was trained using individual neuron reconstructions as the fundamental training units. For each optimization step, all dendritic trees belonging to a single SWC file were processed jointly, allowing neuron-level classification constraints to be incorporated during learning and inference.

S8. Dataset Partitioning

The dataset consisted of curated pyramidal neuron reconstructions containing a single apical dendritic tree.

Neuron reconstructions were randomly divided at the file level into:

- 80% training,
- 10% validation,
- 10% testing.

Splitting at the file level ensured that dendritic trees originating from the same neuron never appeared in multiple subsets, thereby preventing information leakage.

Training, validation, and testing were distributed across MPI processes for parallel execution. Each process independently trained and evaluated its assigned subset, while only summary statistics were gathered at the end of training.

S9. Prediction Constraints

Following network inference, anatomical consistency constraints were applied to the predicted labels.

Trees labeled as **other** in the ground truth remained fixed as **other** during prediction. Among the remaining dendritic trees, the model enforced a biologically consistent single-apical policy by selecting the tree with the largest apical-versus-basal decision margin as the unique apical dendrite, while all remaining non-other trees were classified as basal.

This post-processing step ensured that each neuron reconstruction contained at most one predicted apical dendritic tree, consistent with the biological characteristics of the pyramidal neurons used for training.

S10. Evaluation Metrics

Model performance was evaluated using:

- classification accuracy,
- weighted precision,
- weighted recall,
- weighted F1-score,
- prediction confidence,
- prediction uncertainty.

Confidence scores were computed as the softmax probability assigned to the predicted class,

$$Confidence = \max(\text{Softmax}(z))$$

while uncertainty was defined as

$$Uncertainty = 1 - Confidence.$$

The best-performing model in each training cycle was selected according to validation accuracy and subsequently evaluated on the independent test set.